

Technisches Konzept

UmweltMonitor3000

Technologie: .NET 8 / WPF / MQTT / SQLite | Entwickler: Dazooo | 05.05.2026

1. Systemübersicht

Das System besteht aus drei Hauptkomponenten: Raspberry Pi (Sensor-Node), MQTT-Broker (Mosquitto) und der Desktop-Anwendung (WPF). Diese kommunizieren über MQTT im lokalen Netzwerk.

Komponente	Technologie	Aufgabe
Sensor-Node (Raspberry Pi)	Raspberry Pi OS + Python	Sensordaten erfassen und per MQTT senden
MQTT-Broker	Mosquitto	Nachrichten zwischen Sensor-Node und App vermitteln
Desktop-App	WPF / .NET 8	Daten empfangen, visualisieren, speichern, alarmieren

2. Sensor-Node: Raspberry Pi

2.1 Hardware

- Raspberry Pi (Modell 3B+ oder höher)
- Kapazitiver Bodenfeuchtigkeitssensor (über ADC ausgelesen)
- Temperatursensor (z.B. DHT22 oder DS18B20)
- WLAN-Verbindung zum lokalen Netzwerk

2.2 Software

- Betriebssystem: Raspberry Pi OS (Lite)
- MQTT-Client: paho-mqtt (Python)
- Messung alle 60 Sekunden, Übertragung als JSON-Payload

2.3 MQTT Payload (Beispiel)

Topic: `umweltmonitor/sensor/Tomate`
Payload: `{"moistureSensor": 512, "moisturePercent": 65, "battery": 87}`

3. MQTT-Broker

Parameter	Wert
Software	Eclipse Mosquitto
Standard-Port	1883 (MQTT ohne TLS)
Subscribe-Topic der App	<code>umweltmonitor/sensor/#</code> (Wildcard)
Publish-Topic (Sensor)	<code>umweltmonitor/sensor/{PflanzenName}</code>

4. Desktop-Anwendung

4.1 Architektur (MVVM)

Schicht	Klassen / Komponenten
View	MainWindow.xaml, Converter (BoolToColor, ValueToRangeConverter ...)
ViewModel	MainViewModel, MqttViewModel, PlantViewModel, StatisticViewModel, MainWindowLogic
Model	Plant, SensorData, TopicLog
Service	MqttService (MQTTnet), NotificationService
Repository	UmweltMonitorRepository (SQLite)

4.2 Datenfluss

Schritt	Beschreibung
1	Raspberry Pi misst Sensordaten und sendet MQTT-Nachricht an den Broker
2	MqttService empfängt die Nachricht und löst das MessageReceived-Event aus
3	MainWindowLogic verarbeitet das Event, speichert Daten in SQLite
4	PlantViewModel aktualisiert Pflanze (Feuchtigkeit, Batterie, Zeitstempel)
5	StatisticViewModel aktualisiert Charts und Handlungsempfehlung
6	MqttViewModel fügt Log-Eintrag hinzu, speichert in logs.json
7	NotificationService sendet Windows-Benachrichtigung

4.3 Alarmfunktion / Schwellenwerte

Bereich	Status	Handlungsempfehlung
0 – 30 %	Kritisch	Pflanze sofort gießen! (Rot + Benachrichtigung)
31 – 40 %	Durstig	Feuchtigkeit sinkt! (Gelb + Warnung)
41 – 70 %	Perfekt	Kein Eingriff nötig (Grün)
71 – 100 %	Zu nass	Staunässegefahr! (Blau + Warnung)

5. Datenspeicherung

Datei / DB	Format	Inhalt
plants.json	JSON	Pflanzenliste mit Sensorwerten; bei jeder Änderung gespeichert
logs.json	JSON	Log-Einträge (max. 500) + Fehlerzähler
SQLite-DB	SQLite	Historische Sensordaten (Topic, Value, TimeStamp)

6. Entwicklungsumgebung

Tool	Version / Beschreibung
IDE	Visual Studio 2022
Laufzeit	.NET 8 SDK
Sprache	C# 12
Versionsverwaltung	Git / GitHub
Betriebssystem	Windows 11
MQTT-Broker (Test)	Mosquitto (lokal oder auf Raspberry Pi)

